Python from Zero

A Beginner's Handbook and Cheat Sheet

Variables · Text · Lists · Tuples · Sets · Dictionaries
Loops · Comprehensions · Functions · Files · JSON · CSV

Written to accompany the practice notebook
PracticaOPCIONAL.ipynb

How to Use This Book

This book assumes you have **never programmed before**. It starts with the very first idea (what a program is) and builds up, chapter by chapter, to everything needed for the 17 practice exercises: dictionaries, sets, tuples, loops, text processing, and reading and writing files in several formats.

- **Chapters 1–4** are the foundation: running code, variables, operators, text.
- **Chapters 5–12** cover decisions, loops, and all the data containers (lists, tuples, sets, dictionaries), plus comprehensions and functions.
- **Chapters 13–15** cover files, JSON, and CSV.
- **Chapter 16** collects reusable *recipes*, the patterns you will use again and again.
- **Chapter 17** helps you read error messages.
- **Appendix A** maps each notebook exercise to the chapters you need, with hints (not full solutions). **Appendix B** has answers to the "Try it yourself" boxes. **Appendix C** is a one-stop quick reference.

Tip

The best way to learn is to *type the examples yourself* and change them. Break things on purpose. Error messages are your friends, not your enemies.

Throughout the book, code appears in a shaded block with a blue bar. The output that the code produces appears right below it in a gray block:

```
print("Hello!")
```

```
Hello!
```

Contents

How to Use This Book **2**

1 Getting Started **4**

1.1 What is programming? . . . 4
1.2 Jupyter notebooks . . . 4
1.3 Your first program . . . 4
1.4 Comments . . . 5
1.5 Syntax rules to remember . . . 5

2 Variables and Basic Types **6**

2.1 Variables . . . 6
2.2 The basic types . . . 6
2.3 Converting between types . . . 6

3 Operators and Expressions **8**

3.1 Arithmetic . . . 8
3.2 Comparison operators . . . 8
3.3 Logical operators . . . 8
3.4 Membership: `in` . . . 9

4 Text (Strings) **10**

4.1 Indexing and slicing . . . 10
4.2 Useful string methods . . . 10
4.3 f-strings: mixing text and values . . . 11

5 Making Decisions **12**

5.1 if, elif, else . . . 12
5.2 Truthiness shortcuts . . . 12
5.3 The one-line conditional expression . . . 12
5.4 Order of checks matters . . . 13

6 Lists **14**

6.1 Changing a list . . . 14
6.2 A list of lists, and a list of dictionaries . . . 14

7 Loops **16**

7.1 The for loop . . . 16
7.2 range . . . 16

7.3 Unpacking in a loop . . . 16
7.4 The accumulator pattern . . . 16
7.5 enumerate and zip . . . 17
7.6 while, break, continue . . . 17

8 Tuples **18**
8.1 Unpacking . . . 18
8.2 One-item tuples . . . 18
8.3 Converting . . . 18

9 Sets **20**
9.1 Set operations . . . 20

10 Dictionaries **21**
10.1 Reading, adding, changing, deleting . . . 21
10.2 Looping over a dictionary . . . 21
10.3 Nested dictionaries . . . 22
10.4 List of dictionaries . . . 22
10.5 Counting with a dictionary . . . 22
10.6 Sorting dictionary data . . . 23

11 Comprehensions **24**
11.1 List comprehensions . . . 24
11.2 Replacing items conditionally . . . 24
11.3 Comprehensions over pairs . . . 24
11.4 Dictionary and set comprehensions . . . 25

12 Functions and Useful Built-ins **26**
12.1 Using functions . . . 26
12.2 Defining your own functions . . . 26
12.3 Modules and import . . . 27

13 Working with Files **28**
13.1 Opening files safely: with . . . 28
13.2 Reading . . . 28
13.3 Parsing a line of data . . . 28

14 JSON **30**
14.1 Writing and reading . . . 30
14.2 What JSON can (and cannot) hold . . . 30

15 CSV Files **32**
15.1 Writing . . . 32

15.2 Reading 32
15.3 Writing dictionaries 32
15.4 Packing several values into one cell 33

16 Recipes: Patterns You Will Reuse **34**
16.1 “Convert / transform every item” 34
16.2 “Keep only the items where…” 34
16.3 “Add up / total / how many” 34
16.4 “Count how often each… appears” 34
16.5 “Group items by…” 34
16.6 “Collect all unique… across everything” 35
16.7 “Is this number even / a multiple of…?” 35
16.8 “Difference between the biggest and smallest” 35
16.9 “Sort by frequency / by a field” 35
16.10 “Average, maximum and minimum of a list” 35
16.11 “Count the words in a text” 35
16.12 “Read a file, process it, write the result” 36

17 Errors and Debugging **37**
17.1 A debugging routine 37
17.2 Common beginner pitfalls 37

A Exercise Map and Hints **39**

B Answers to “Try it yourself” **42**

C Quick Reference **43**

Chapter 1

Getting Started

1.1 What is programming?

A **program** is a list of precise instructions that a computer follows, one after another. Computers are extremely fast but completely literal: they do exactly what you write, never what you meant. Programming is the skill of describing a task so clearly that a literal-minded machine can carry it out.

Python is a programming language, a set of words and rules for writing those instructions. It is popular because its code reads almost like English.

1.2 Jupyter notebooks

Your assignment is a **Jupyter notebook** (a `.ipynb` file). A notebook is a document made of **cells**:

- **Markdown cells** contain text, like the exercise statements (the ones with `####` headings).
- **Code cells** contain Python. Select a code cell and press `Shift+Enter` to *run* it. The result appears below the cell.

The notebook keeps a running Python session called the **kernel**. Everything you define in one cell (variables, for instance) stays available in later cells.

Watch out

Cells remember, and run order matters. If a cell changes a variable (say, adds an item to a list) and you run it twice, the change happens twice. If something looks strange, use the menu *Kernel* → *Restart & Run All* to start fresh and run everything top to bottom.

1.3 Your first program

`print()` shows something on the screen. Whatever you put between the parentheses is displayed.

```
print("Hello, world!")
print(2 + 3)
print("Total:", 10)
```

```
Hello, world!
5
Total: 10
```

In a notebook, if the *last line* of a cell is just a value, Jupyter shows it automatically, even without `print`. Use `print` when you want to display several things or control the format.

1.4 Comments

A **comment** is a note for humans. Python ignores everything after a `#` on a line.

```
# This line is a comment and does nothing
print("Visible")    # a comment can also follow code
```

```
Visible
```

1.5 Syntax rules to remember

- Python is **case-sensitive**: `name` and `Name` are different things.
- **Indentation matters.** Blocks of code are marked by indenting with 4 spaces (more in Chapter 5).
- Text must be inside quotes: `"like this"` or `'like this'`.
- Parentheses, brackets, and quotes must be **closed**: every opening has a matching closing one.

Chapter 2

Variables and Basic Types

2.1 Variables

A **variable** is a name attached to a value, like a labelled box. You create one with the **assignment** operator `=`. Read it as "the name on the left now refers to the value on the right".

```
name = "Teo"
age = 27
print(name, age)

age = age + 1        # take the old value, add 1, store it back
print(age)
```

```
Teo 27
28
```

Naming rules: letters, digits and underscores only; cannot start with a digit; no spaces. By convention use lowercase with underscores: `total_points`. Pick names that say what the value means.

2.2 The basic types

Every value has a **type**, which determines what you can do with it.

Type	Name	Meaning	Examples
`int`	integer	whole number	`27`, `-3`, `0`
`float`	float	decimal number	`3.14`, `-0.5`
`str`	string	text	`"hello"`, `'a'`
`bool`	boolean	true or false	`True`, `False`
`NoneType`	none	"nothing here"	`None`

`type(x)` tells you the type of `x`:

```
print(type(27), type(3.14), type("hi"), type(True))
```

```
<class 'int'> <class 'float'> <class 'str'> <class 'bool'>
```

Watch out

`True` and `False` are capitalised and have *no quotes*. `"True"` in quotes is just text, not a boolean.

2.3 Converting between types

```
print(int("42") + 1)      # text to integer -> 43
```

```
print(str(42) + "!")       # integer to text -> 42!
print(float("2.5"))        # text to float   -> 2.5
print(int(3.9))            # float to int cuts the decimals -> 3
```

```
43
42!
2.5
3
```

This matters a lot when reading files: everything you read from a file is *text*, so numbers must be converted with `int()` before you can do arithmetic.

Try it yourself (answers in Appendix B)

1. What is `type(3.0)`?
2. What does `"5" + "5"` produce?
3. What does `int("5") + int("5")` produce?

Chapter 3

Operators and Expressions

An **expression** is anything Python can calculate to a value. **Operators** combine values.

3.1 Arithmetic

Operator	Meaning	Example → result
`+`	addition	`7 + 2` → `9`
`-`	subtraction	`7 - 2` → `5`
`*`	multiplication	`7 * 2` → `14`
`/`	division (always a float)	`7 / 2` → `3.5`
`//`	whole-number division	`7 // 2` → `3`
`%`	remainder (*modulo*)	`7 % 2` → `1`
`**`	power	`7 ** 2` → `49`

Key idea

The modulo operator `%` is the key to "is it even?" and "is it a multiple of n?" questions. `x % 2 == 0` means `x` is even. `x % 3 == 0` means `x` is a multiple of 3. If the remainder after dividing is 0, there is nothing left over, so it divides evenly.

3.2 Comparison operators

These produce a boolean (`True` or `False`).

Operator	Meaning
`==`	equal to
`!=`	not equal to
`<`, `>`	less than, greater than
`<=`, `>=`	less or equal, greater or equal

Watch out

A single `=` **stores** a value. A double `==` **asks** whether two values are equal. Mixing them up is the most common beginner mistake.

3.3 Logical operators

Combine booleans with `and`, `or`, `not`:

```
age = 27
active = True
```

```
print(age > 18 and active)      # both must be true
print(age < 18 or active)       # at least one true
print(not active)               # flips it
```

```
True
True
False
```

3.4 Membership: in

`x in collection` asks "is `x` inside?" It works for strings, lists, tuples, sets, and (for keys) dictionaries.

```
print("a" in "banana")
print(3 in [1, 2, 3])
print("Perl" in ["Python", "Java"])
```

```
True
True
False
```

Try it yourself (answers in Appendix B)

1. What is `17 % 5`?
2. What is `17 // 5`?
3. What is `3 > 2 and not (1 == 2)`?

Chapter 4

Text (Strings)

A **string** is a sequence of characters. Strings are **immutable**: you can never change one in place; methods return a *new* string.

4.1 Indexing and slicing

Positions are numbered from **0**. Negative numbers count from the end.

`s`	b	a	n	a	n	a
index	0	1	2	3	4	5
negative	-6	-5	-4	-3	-2	-1

```
s = "banana"
print(s[0])        # first character
print(s[-1])       # last character
print(s[1:4])      # from index 1 up to (not including) 4
print(len(s))      # length
```

```
b
a
ana
6
```

Slice syntax is `s[start:stop]`: it includes `start` but *excludes* `stop`.

4.2 Useful string methods

A **method** is a function attached to a value, called with a dot: `value.method()`.

Method	What it does
`s.upper()`	ALL CAPITALS
`s.lower()`	all lowercase
`s.capitalize()`	First letter capital, the rest lowercase
`s.title()`	Every Word Capitalised
`s.strip()`	remove spaces/newlines at both ends
`s.replace(a, b)`	replace every `a` with `b`
`s.split(sep)`	cut into a *list* of pieces at each `sep`
`sep.join(list)`	glue a list of strings together using `sep`
`s.count(x)`	how many times `x` appears
`s.startswith(x)`	does it begin with `x`?

```
print("ana".upper())
print("mARTA".capitalize())
print("  padded  ".strip())
```

```
print("a-b-c".replace("-", "+"))
print("one two three".split())        # no argument: split on spaces
print("x,y,z".split(","))
print(" | ".join(["red", "green"]))
```

```
ANA
Marta
padded
a+b+c
['one', 'two', 'three']
['x', 'y', 'z']
red | green
```

Watch out

Because strings are immutable, `name.upper()` *alone* does nothing useful. You must **store** the result: `name = name.upper()`.

4.3 f-strings: mixing text and values

Put an `f` before the opening quote and write values inside curly braces:

```
name = "Ana"
avg = 20.0
print(f"{name} has an average of {avg}")
```

```
Ana has an average of 20.0
```

Try it yourself (answers in Appendix B)

1. What does `"hello world".upper()` give?
2. What does `"hello"[1:3]` give?
3. What does `"a,b".split(",")` give?

Chapter 5

Making Decisions

5.1 if, elif, else

Programs often need to do different things in different situations. The `if` statement runs a block of code only when a condition is `True`.

```
score = 15

if score >= 20:
    print("pass")
elif score >= 10:
    print("almost")
else:
    print("fail")
```

```
almost
```

Key idea

Indentation is the structure. The colon `:` at the end of the `if` line opens a block; the indented lines below (4 spaces) belong to it. Python checks the conditions from top to bottom and runs only the *first* block whose condition is true. `elif` means "else, if" and `else` catches everything else. Both are optional.

5.2 Truthiness shortcuts

A boolean variable can be used directly as a condition:

```
active = True
if active:                  # same as: if active == True:
    print("on")
```

```
on
```

5.3 The one-line conditional expression

When you just need to choose between two values:

```
n = 7
label = "even" if n % 2 == 0 else "odd"
print(label)
```

```
odd
```

The shape is `VALUE_IF_TRUE if CONDITION else VALUE_IF_FALSE`. You will use this inside comprehensions (Chapter 11).

5.4 Order of checks matters

When conditions overlap, put the **most specific** one first. If you test "is it even?" before "is it even *and* a multiple of 4?", the second test is never reached, since any multiple of 4 already matched the first.

Try it yourself (answers in Appendix B)

1. With `x = 7`, what does `if x % 2 == 0: print("even") else: print("odd")` print?
2. With `x = 15`, what is printed by an `if x > 10 / elif x > 5 / else` chain that prints `"big"`, `"medium"`, `"small"` respectively?

Chapter 6

Lists

A **list** is an ordered, changeable collection written with square brackets. Items can be of any type and can repeat.

```
skills = ["Python", "SQL", "Java"]
print(skills[0])        # indexing works like strings
print(skills[-1])
print(len(skills))
print("SQL" in skills)
```

```
Python
Java
3
True
```

6.1 Changing a list

Lists are **mutable**: you can modify them in place.

```
nums = [5, 3, 8]
nums.append(1)          # add to the end
nums[0] = 99            # replace an item
nums.remove(3)          # remove the first 3
print(nums)
```

```
[99, 8, 1]
```

Operation	Effect
`lst.append(x)`	add `x` at the end
`lst.insert(i, x)`	insert `x` at position `i`
`lst.remove(x)`	delete the first `x`
`lst.pop()`	remove and return the last item
`lst.sort()`	sort *in place* (returns nothing)
`sorted(lst)`	return a *new* sorted list
`lst[a:b]`	slice, like strings
`lst1 + lst2`	join two lists into a new one

Watch out

`lst.sort()` and `lst.append(x)` change the list and return `None`. Writing `lst = lst.sort()` wipes out your list! In contrast, `sorted(lst)` leaves the original alone and gives you a new list.

6.2 A list of lists, and a list of dictionaries

Containers can hold other containers. This is how real data is usually organised:

```
grid = [[1, 2], [3, 4]]
print(grid[1][0])       # second row, first item
```

```
3
```

Try it yourself (answers in Appendix B)

Let `nums = [4, 8, 15]`.

1. What is `nums[-1]`?
2. What is `nums[:2]`? (An omitted start means "from the beginning".)
3. After `nums.append(16)`, what is `len(nums)`?

Chapter 7

Loops

A **loop** repeats a block of code. It is how you process every item in a collection without writing the same line a hundred times.

7.1 The for loop

```
for skill in ["Python", "SQL", "Java"]:
    print("I know", skill)
```

```
I know Python
I know SQL
I know Java
```

On each pass, the variable (`skill`) takes the next item. The indented block runs once per item.

7.2 range

`range(n)` produces the numbers $0, 1, \ldots, n-1$. `range(a, b)` produces $a, \ldots, b-1$.

```
for i in range(3):
    print(i)
print(list(range(1, 5)))
```

```
0
1
2
[1, 2, 3, 4]
```

7.3 Unpacking in a loop

If each item is a pair (a tuple), you can split it into two variables right in the `for` line. This is extremely useful for lists of tuples.

```
pairs = [(1, 5), (2, 2), (10, 3)]
for a, b in pairs:
    print(a, "vs", b)
```

```
1 vs 5
2 vs 2
10 vs 3
```

7.4 The accumulator pattern

Start with a neutral value, then update it on every pass.

```
total = 0
```

```
for n in [4, 8, 15]:
    total = total + n       # shorthand: total += n
print(total)
```

```
27
```

7.5 enumerate and zip

```
for i, name in enumerate(["Ana", "Luis"]):
    print(i, name)
for name, score in zip(["Ana", "Luis"], [10, 20]):
    print(name, score)
```

```
0 Ana
1 Luis
Ana 10
Luis 20
```

7.6 while, break, continue

`while` repeats as long as a condition stays true. `break` exits a loop early; `continue` skips to the next pass.

```
count = 0
while count < 3:
    count += 1
print(count)
```

```
3
```

Watch out

A `while` loop whose condition never becomes false runs forever. If a notebook cell seems stuck (`[*]` next to it), use *Kernel* → *Interrupt*.

Try it yourself (answers in Appendix B)

1. How many times does `for i in range(3):` run its body?
2. What does the accumulator pattern give for the numbers $1, 2, 3, 4$?

Chapter 8

Tuples

A **tuple** is like a list, but **immutable**: once created, it cannot be changed. It is written with round brackets.

```
point = (3, 4)
print(point[0])
print(len(point))
```

```
3
2
```

```
point[0] = 9
```

```
TypeError: 'tuple' object does not support item assignment
```

8.1 Unpacking

```
x, y = (3, 4)
print(x, y)
```

```
3 4
```

8.2 One-item tuples

A tuple with one item needs a trailing comma: `(5,)`. Without it, `(5)` is just the number 5 in parentheses.

8.3 Converting

```
t = (3, 1, 2)
print(list(t))              # tuple to list
print(tuple([1, 2, 3]))     # list to tuple
print(sorted(t))            # note: sorted always returns a LIST
print(tuple(sorted(t)))     # so wrap it to get a tuple back
```

```
[3, 1, 2]
(1, 2, 3)
[1, 2, 3]
(1, 2, 3)
```

Key idea

Almost every function that processes a collection (`sorted`, `set`, `list`, `tuple`) **accepts any collection** and returns a specific type. To change the type of a collection, wrap it in the constructor of the target type.

Try it yourself (answers in Appendix B)

With `t = (1, 2, 3)` and `a, b, c = t`, what is `b`? What error does `t[0] = 9` cause?

Chapter 9

Sets

A **set** is an **unordered** collection of **unique** items, written with curly braces. Duplicates are discarded automatically.

```
s = {1, 2, 2, 3, 3, 3}
print(s)
print(set([5, 5, 7]))        # build a set from any collection
print(set("banana"))         # a string is a collection of characters
```

```
{1, 2, 3}
{5, 7}
{'a', 'b', 'n'}
```

Watch out

Sets have no order and no indexing. `s[0]` is an error, and the order in which items print is not guaranteed (the output of `set("banana")` could be shown in a different order on your machine). Also, `{}` is an empty *dictionary*, not an empty set; use `set()` for an empty set.

9.1 Set operations

```
a = {1, 2, 3}
b = {2, 3, 4}
a.add(10)                    # add one item
print(a | b)                 # union: everything
print(a & b)                 # intersection: in both
print(a - b)                 # difference: in a but not in b
print(len(b), 3 in b)
```

```
{1, 2, 3, 4, 10}
{2, 3}
{1, 10}
3 True
```

`a.update(other)` adds all items from another collection into `a`, which is handy for building a "set of everything" from many smaller sets.

Key idea

Use a set whenever you hear **"unique"**, "without repeats", or "all distinct values". Converting a list to a set is the standard way to remove duplicates.

Try it yourself (answers in Appendix B)

1. What is `set([1, 2, 2, 3, 3, 3])`?
2. What is `{1, 2, 3} & {2, 3, 4}`?

Chapter 10

Dictionaries

A **dictionary** stores **key** → **value** pairs. You look up a value by its key, like finding a word in a real dictionary.

```
person = {"name": "Ana", "age": 30, "skills": ["Python", "SQL"]}
print(person["name"])
print(person["skills"][1])
```

```
Ana
SQL
```

Keys are usually strings (or numbers). Values can be anything, including lists, sets, or other dictionaries.

10.1 Reading, adding, changing, deleting

```
person["city"] = "Rome"           # add a new key
person["age"] = 31                # change an existing one
del person["skills"]              # remove a key
print(person)
print(person.get("phone"))        # safe lookup: None if missing
print(person.get("phone", "n/a"))# ... or a default
print("city" in person)           # tests KEYS
```

```
{'name': 'Ana', 'age': 31, 'city': 'Rome'}
None
n/a
True
```

Watch out

`person["phone"]` with a missing key raises a `KeyError`. Use `.get()` when the key might not exist.

10.2 Looping over a dictionary

```
ages = {"ana": 30, "luis": 25}
for key in ages:                      # keys only
    print(key)
for key, value in ages.items():       # keys and values together
    print(key, "->", value)
print(list(ages.keys()), list(ages.values()))
```

```
ana
luis
ana -> 30
luis -> 25
['ana', 'luis'] [30, 25]
```

10.3 Nested dictionaries

Dictionaries often hold dictionaries. Chain the lookups:

```
users = {
    "u1": {"name": "ana", "roles": {"admin", "user"}},
    "u2": {"name": "luis", "roles": {"user"}},
}
print(users["u1"]["name"])
users["u2"]["name"] = users["u2"]["name"].capitalize()
print(users["u2"]["name"])

for uid, info in users.items():          # uid = key, info = inner dict
    print(uid, info["name"])
```

```
ana
Luis
u1 ana
u2 Luis
```

Key idea

When you loop with `for uid, info in users.items()`, `info` is the *actual* inner dictionary, not a copy. Changing `info["name"] = ...` inside the loop updates the original `users`. But reassigning a plain string variable (`name = name.upper()`) only changes that variable, not the dictionary.

10.4 List of dictionaries

Another very common shape (a table of records):

```
records = [
    {"id": 1, "name": "ana"},
    {"id": 2, "name": "luis"},
]
for r in records:
    r["name"] = r["name"].capitalize()     # modifies the record in place
    r["done"] = True                       # adds a new key to each record
print(records)
```

```
[{'id': 1, 'name': 'Ana', 'done': True}, {'id': 2, 'name': 'Luis', 'done': True}]
```

10.5 Counting with a dictionary

```
counts = {}
for ch in "hello":
    counts[ch] = counts.get(ch, 0) + 1
print(counts)
```

```
{'h': 1, 'e': 1, 'l': 2, 'o': 1}
```

Read it as: “look up the current count of this character (or 0 if unseen), add one, store it back”.

10.6 Sorting dictionary data

`counts.items()` gives key–value pairs as tuples. `sorted` can sort them by any rule you give through its `key` argument. A **lambda** is a tiny nameless function: `lambda pair: pair[1]` means "given a pair, return its second element".

```
ranked = sorted(counts.items(), key=lambda pair: pair[1], reverse=True)
print(ranked)
```

```
[('l', 2), ('h', 1), ('e', 1), ('o', 1)]
```

Try it yourself (answers in Appendix B)

Start with `d = {"a": 1}`.

1. After `d["b"] = 2`, what is `d`?
2. What does `d.get("z", 0)` return?

Chapter 11

Comprehensions

A **comprehension** builds a new collection from an existing one in a single line. Once you can read them, they replace many small loops.

11.1 List comprehensions

The shape is `[EXPRESSION for ITEM in COLLECTION]`, optionally followed by `if CONDITION`.

```
nums = [1, 2, 3, 4, 5, 6]

doubled = [n * 2 for n in nums]
evens   = [n for n in nums if n % 2 == 0]
labels  = ["even" if n % 2 == 0 else "odd" for n in nums]

print(doubled)
print(evens)
print(labels)
```

```
[2, 4, 6, 8, 10, 12]
[2, 4, 6]
['odd', 'even', 'odd', 'even', 'odd', 'even']
```

Key idea

Two different roles for `if`:

- `[x for x in xs if cond]` *filters*: items failing the condition are **dropped** (the output can be shorter).
- `[A if cond else B for x in xs]` *transforms*: every item stays, but becomes `A` or `B`. The output has the **same length**.

11.2 Replacing items conditionally

```
fruits = ["apple", "kiwi", "plum"]
fruits = ["pear" if f == "kiwi" else f for f in fruits]
print(fruits)
```

```
['apple', 'pear', 'plum']
```

11.3 Comprehensions over pairs

```
pairs = [(1, 5), (10, 3), (4, 4)]
bigger_first = [(a, b) for a, b in pairs if a > b]
sums         = [a + b for a, b in pairs]
even_sums    = [(a + b) % 2 == 0 for a, b in pairs]
print(bigger_first, sums, even_sums)
```

```
[(10, 3)] [6, 13, 8] [True, False, True]
```

11.4 Dictionary and set comprehensions

```
words = ["hi", "hello", "yo"]
lengths = {w: len(w) for w in words}          # {key: value for ...}
unique_lengths = {len(w) for w in words}      # {value for ...}  (a set)
print(lengths)
print(unique_lengths)
```

```
{'hi': 2, 'hello': 5, 'yo': 2}
{2, 5}
```

You can also filter: `{k: v for k, v in counts.items() if v > 1}` keeps only the entries whose value exceeds 1.

Tip

If a comprehension gets hard to read, write it as a normal `for` loop first, get it working, then compress it.

Try it yourself (answers in Appendix B)

1. What is `[x * x for x in range(4)]`?
2. What is `[x for x in range(6) if x > 3]`?

Chapter 12

Functions and Useful Built-ins

12.1 Using functions

A **function** is a named, reusable piece of code. You *call* it with parentheses, passing **arguments**, and it may **return** a result. Python ships with many built-in functions:

Function	Result
`len(x)`	number of items
`sum(x)`	total of the numbers
`min(x)`, `max(x)`	smallest, largest
`sorted(x)`	new sorted list
`sorted(x, reverse=True)`	descending order
`abs(x)`	absolute value
`round(x, n)`	round to `n` decimals
`any(x)`, `all(x)`	is at least one / every item true?
`list(x)`, `tuple(x)`, `set(x)`	convert to that type
`int(x)`, `float(x)`, `str(x)`	convert a single value
`range(n)`	numbers $0 \ldots n-1$

```
scores = [10, 20, 30]
average = sum(scores) / len(scores)
print(average, max(scores), min(scores))
print(round(7 / 3, 2))
print(sorted([3, 1, 2], reverse=True))
```

```
20.0 30 10
2.33
[3, 2, 1]
```

Watch out

Dividing by `len(x)` fails if the list is empty (`ZeroDivisionError`). All the lists in this assignment are non-empty, but remember it for real data.

12.2 Defining your own functions

```
def describe(name, score=0):
    """Return a short sentence about a student."""
    return f"{name} scored {score}"

print(describe("Ana", 20))
print(describe("Luis"))
```

```
Ana scored 20
Luis scored 0
```

- `def` starts the definition; the indented block is the body.
- `score=0` is a **default value**, used when the caller omits that argument.
- `return` sends a value back. A function without `return` gives back `None`.

12.3 Modules and import

Some tools live in **modules**, which you load with `import`. This assignment uses `json`, `csv`, and (optionally) `re` and `collections`.

```
import json
from collections import Counter
```

Try it yourself (answers in Appendix B)

1. What is `max([3, 9, 2])`?
2. What is `sorted([3, 1, 2], reverse=True)`?
3. What is `round(sum([1, 2, 4]) / 3, 2)`?

Chapter 13

Working with Files

Variables disappear when you close Python. **Files** let data survive. A text file is a sequence of characters, organised in lines separated by the invisible newline character `\n`.

13.1 Opening files safely: with

```
with open("greetings.txt", "w", encoding="utf-8") as f:
    f.write("Hello\n")
    f.write("World\n")
```

- `open(name, mode)` gives you a file object (here called `f`).
- `with ... as f:` guarantees the file is closed properly when the indented block ends. **Always use it.**
- `encoding="utf-8"` makes accents and characters like `ñ` work correctly. Use it whenever you read or write text.

Mode	Meaning
`"r"`	**r**ead (default); the file must exist
`"w"`	**w**rite; creates the file or **erases** an existing one
`"a"`	**a**ppend; add to the end without erasing

13.2 Reading

```
with open("greetings.txt", "r", encoding="utf-8") as f:
    text = f.read()                 # the whole file as one string
print(repr(text))

with open("greetings.txt", "r", encoding="utf-8") as f:
    for line in f:                  # one line at a time
        print(line.strip())         # strip removes the trailing \n
```

```
'Hello\nWorld\n'
Hello
World
```

13.3 Parsing a line of data

A very common job: each line holds comma-separated fields. Split, then convert the numbers.

```
line = "Bob,7,8,9\n"
parts = line.strip().split(",")        # ['Bob', '7', '8', '9']
name = parts[0]
scores = [int(p) for p in parts[1:]]   # everything after the name, as ints
print(name, scores)
```

```
Bob [7, 8, 9]
```

Watch out

Where is my file? Relative names like `"greetings.txt"` refer to the notebook's *current folder*. Files you write appear next to the notebook, and a file you want to read (such as `fragmento-libro.txt` in Exercise 15) must be in that same folder. If you get `FileNotFoundError`, check the name and the folder. `import os; os.getcwd()` prints the current folder.

Watch out

Mode `"w"` **silently destroys** the previous contents. If you only want to add lines, use `"a"`.

Chapter 14

JSON

JSON (JavaScript Object Notation) is a text format for storing structured data, and almost every programming language and web service understands it. It looks very much like Python dictionaries and lists:

```
[
  {"name": "Ana", "scores": [10, 20, 30], "active": true},
  {"name": "Luis", "scores": [5, 15], "active": false}
]
```

14.1 Writing and reading

```
import json

profile = {"name": "Bob", "scores": [7, 8, 9], "active": True}

with open("profile.json", "w", encoding="utf-8") as f:
    json.dump(profile, f, indent=2, ensure_ascii=False)

with open("profile.json", "r", encoding="utf-8") as f:
    loaded = json.load(f)

print(loaded["name"], loaded["scores"])
```

```
Bob [7, 8, 9]
```

- `json.dump(obj, f)`: Python object → file. Option `indent=2` makes it human-readable.
- `json.load(f)`: file → Python object.
- `ensure_ascii=False` keeps accented letters readable rather than escaping them.

14.2 What JSON can (and cannot) hold

Python	JSON
`dict`	object `{...}`
`list`, `tuple`	array `[...]`
`str`	string
`int`, `float`	number
`True`, `False`	`true`, `false`
`None`	`null`

Watch out

Sets cannot be saved to JSON. Dumping a set raises `TypeError: Object of type set is not JSON serializable`. Convert first: `sorted(my_set)` or `list(my_set)`. Also,

tuples come back as lists after a save-and-load round trip, and dictionary **keys always become strings**.

```
data = {"roles": sorted({"user", "admin"})}      # set -> sorted list
print(json.dumps(data))                          # dumps = to a string
print(json.loads('{"x": [1, 2]}'))               # loads = from a string
```

```
{"roles": ["admin", "user"]}
{'x': [1, 2]}
```

Chapter 15

CSV Files

CSV (comma-separated values) is a plain-text table: one row per line, columns separated by commas. Spreadsheets like Excel open it directly.

```
product,quantity
A,10
B,5
```

15.1 Writing

```python
import csv

rows = [["item", "qty"], ["pen", 3], ["book", 1]]

with open("stock.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerows(rows)          # many rows at once
    # writer.writerow(["x", 1])     # ...or one row at a time
```

`newline=""` is required when using the `csv` module (it avoids blank lines on Windows).

15.2 Reading

`csv.DictReader` turns each row into a dictionary using the header line as keys. This is the friendliest way to read.

```python
with open("stock.csv", "r", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)          # first line = header = the keys
    for row in reader:
        print(row["item"], int(row["qty"]))
```

```
pen 3
book 1
```

Watch out

Everything read from a CSV is a string. `row["qty"]` is `"3"`, not `3`. Convert with `int()` before adding numbers, otherwise `"3" + "3"` gives `"33"`.

15.3 Writing dictionaries

```python
records = [{"id": 1, "name": "ANA"}, {"id": 2, "name": "LUIS"}]
with open("people.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["id", "name"])
    writer.writeheader()
    writer.writerows(records)
```

15.4 Packing several values into one cell

A cell holds one piece of text. To store several roles in a single cell, join them with a separator that is not a comma, such as `|`:

```
roles = {"user", "admin"}
cell = "|".join(sorted(roles))
print(cell)
```

```
admin|user
```

`join` needs a collection of *strings*. It accepts a set directly, but order in a set is not guaranteed, so use `sorted(...)` when you want a stable result.

Chapter 16

Recipes: Patterns You Will Reuse

Most exercises are combinations of a handful of patterns. Learn to recognise them from the wording of the task.

16.1 "Convert / transform every item"

Use a comprehension:

```
names = ["ana", "luis"]
names = [n.capitalize() for n in names]
```

16.2 "Keep only the items where..."

Use a filtering comprehension:

```
pairs = [(1, 5), (10, 3), (4, 4)]
kept = [(a, b) for a, b in pairs if a > b]      # [(10, 3)]
```

16.3 "Add up / total / how many"

Use `sum`, `len`, or an accumulator loop.

```
total = sum((10, 20, 30))      # 60
```

16.4 "Count how often each... appears"

Use a counting dictionary, or `collections.Counter`:

```
from collections import Counter

c = Counter("banana")
print(c)
print(c.most_common(2))        # the 2 most frequent, as (item, count) tuples
```

```
Counter({'a': 3, 'n': 2, 'b': 1})
[('a', 3), ('n', 2)]
```

16.5 "Group items by..."

Build a dictionary whose values are lists. `setdefault(key, [])` creates the empty list the first time a key is seen:

```
pairs = [("apple", "red"), ("cherry", "red"), ("lime", "green")]
```

```
groups = {}
for fruit, colour in pairs:
    groups.setdefault(colour, []).append(fruit)
print(groups)
```

```
{'red': ['apple', 'cherry'], 'green': ['lime']}
```

16.6 “Collect all unique... across everything”

Start with an empty set and `update` it, or use a set comprehension:

```
teams = {"t1": {"a", "b"}, "t2": {"b", "c"}}
everyone = set()
for members in teams.values():
    everyone.update(members)
print(sorted(everyone))
```

```
['a', 'b', 'c']
```

16.7 “Is this number even / a multiple of...?”

Use the remainder: `n % 2 == 0`, `n % 3 == 0`. A list of such answers is a comprehension of booleans: `[x % 3 == 0 for x in nums]`.

16.8 “Difference between the biggest and smallest”

`max(group) - min(group)` for each group (tuple, list...).

16.9 “Sort by frequency / by a field”

`sorted(items, key=lambda x: x[1], reverse=True)` sorts descending by the second element.

16.10 “Average, maximum and minimum of a list”

```
notes = [10, 20, 30]
stats = {"promedio": sum(notes) / len(notes), "max": max(notes), "min": min(notes)}
print(stats)
```

```
{'promedio': 20.0, 'max': 30, 'min': 10}
```

16.11 “Count the words in a text”

Lowercase, remove punctuation, split into words, then count. A regular expression extracts the word characters in one go:

```
import re
from collections import Counter

text = "The cat and the hat, and the bat!"
words = re.findall(r"\w+", text.lower())      # letters/digits only
```

```
print(words)
print(Counter(words).most_common(3))
```

```
['the', 'cat', 'and', 'the', 'hat', 'and', 'the', 'bat']
[('the', 3), ('and', 2), ('cat', 1)]
```

`r"\w+"` means "one or more word characters in a row"; it automatically ignores commas, full stops, and exclamation marks and works with accented letters.

16.12 "Read a file, process it, write the result"

Almost every file exercise follows this skeleton:

```
import json

results = []
with open("input.txt", "r", encoding="utf-8") as f:        # 1. read
    for line in f:
        parts = line.strip().split(",")
        scores = [int(p) for p in parts[1:]]
        results.append({"name": parts[0].capitalize(),     # 2. process
                        "avg": sum(scores) / len(scores)})

with open("output.json", "w", encoding="utf-8") as f:      # 3. write
    json.dump(results, f, indent=2)
```

Chapter 17

Errors and Debugging

Errors are normal: professional programmers see them all day. When Python hits a problem it prints a **traceback**. *Read it from the bottom up*: the last line names the error type and explains it; the lines above show where it happened.

Error	Usual cause
`SyntaxError`	typo in the code structure: missing colon, quote, bracket
`IndentationError`	block not indented consistently
`NameError`	using a variable that does not exist (typo, or you did not run the cell that defines it)
`TypeError`	wrong kind of value: `"5" + 5`, changing a tuple, dumping a set to JSON
`ValueError`	right type, bad value: `int("abc")`
`KeyError`	dictionary key not found; use `.get()` or check spelling
`IndexError`	position beyond the end of a list
`FileNotFoundError`	wrong file name or wrong folder
`ZeroDivisionError`	dividing by zero, e.g. the average of an empty list

17.1 A debugging routine

1. **Read** the last line of the error message. Do not panic.
2. **Print** the variables involved: `print(x, type(x))` reveals most surprises (a string that should be an int, a list that should be a dict).
3. **Test small pieces** in a fresh cell. Try the problematic expression on a single item.
4. **Restart and run all** if state seems confusing (Kernel menu).
5. Work **one step at a time**: for multi-step exercises, finish and print the result of step 1 before starting step 2.

17.2 Common beginner pitfalls

- Forgetting to **store** the result: `name.upper()` alone changes nothing; write `name = name.upper()`.
- Using `=` instead of `==` in conditions.
- Forgetting the colon `:` after `if`, `for`, `def`, `while`, `else`.
- Mixing tabs and spaces. Use 4 spaces (the notebook does this for you when you press `Tab`).
- Running a cell that modifies data more than once (appending twice, etc.).
- Reading numbers from files and forgetting that they arrive as text.
- Expecting a set or dictionary to keep a particular order of items.

- Running out of cells' order: using a variable in a cell above where it is created.

Tip

Whenever you are unsure what a command does, try it on a tiny example in a scratch cell. Experiments are free, and `print` is your microscope.

Chapter A

Exercise Map and Hints

This appendix connects each exercise in *PracticaOPCIONAL.ipynb* to the relevant chapters, and gives **hints** (not complete solutions) and notes on a few places where the wording of the assignment is ambiguous.

Ex.	Main skills	Chapters
1	dict access, `if`, `.upper()`, list comprehension with replace, `in`	2–6, 10, 11
2	tuple → set → tuple, `sum`, `sorted`	8, 9, 12
3	counting dict, converting to list of tuples, sorting by value	10, 12
4	filtering pairs, parity as booleans	3, 11
5	nested dict, `.capitalize()`, set size, `.add`, union of sets	4, 9, 10
6	`.split()`, set, counting, filtering a dict	4, 9, 10, 11
7	`max`/`min`, multiples via `%`	3, 11, 12
8	`.upper()`, set of roles, grouping into lists	9, 10, 16
9	`%`, `if/elif/else`, f-strings	3, 5, 11
10	list of dicts, `sum` of a tuple, adding keys, set of names	9, 10
11	`if/elif/else` per pair, comprehension	5, 11
12	dict comprehension or loop, average/-max/min, nested dict, tuple	10, 12, 16
13	write text file, parse lines, average, JSON dump	13, 14, 16
14	JSON write/read, sets in JSON, CSV with `|`	14, 15
15	read text, clean words, count, JSON dump	13, 14, 16
16	write 3 CSV files, read, accumulate, write total	10, 15
17	JSON write/read, dict of dicts	14, 16

Hints

Exercise 1
Take the three steps in order and run them once (re-running the cell on already-modified data gives different results). Check the `"activo"` flag *before* you change anything else, as the task states.

Exercise 2
`set(t)` removes duplicates; `sum()` of the set gives the unique total. `sorted` returns a *list*, so wrap the result in `tuple()`. Use `reverse=True` for largest first.

Exercise 3
Counting recipe from Chapter 10. `sorted(d.items(), key=..., reverse=True)` already returns a list of `(letter, frequency)` tuples.

Exercise 4
Two steps: first filter pairs where the first number is larger, then compute `(a + b) % 2 == 0` for each of the survivors.

Exercise 5
Loop over `usuarios.values()` (each value is an inner dictionary). Capitalise the name; if `len(roles) > 1`, add `"vip"`. For "all roles in the system", start with an empty set and `update` it with each user's roles.

Exercise 6
The sentence contains punctuation and capital letters (`"En"` vs `"en"`, `"camino,"` vs `"camino"`). Decide how to normalise first (lowercase, strip punctuation, see the word-count recipe), then count, then keep entries with count above 1.

Exercise 7
Per tuple: `max(t) - min(t)` and `sum(t) % 3 == 0`. Two separate comprehensions work well.

Exercise 8
Uppercase the names in place; then build a dictionary `role` → `[names]`. The grouping recipe with `setdefault` fits perfectly.

Exercise 9
The text says "multiple of 4" but the label is `"mult3"`. This looks like a typo in the statement; follow the output strings exactly as written, and clarify with your instructor which divisor to test. Whichever it is, **test the "both" case first** (Chapter 5), then the single cases, then fall back to the plain number. Note that the expected output for numbers matching *no* rule is not specified, so decide (for instance, keep the number as a string).

Exercise 10
`.capitalize()` for "first letter in capitals". Each record's `"puntos"` is a tuple, so `sum()` works on it. Add `"total"` and a boolean `"aprobado"` (`total > 50`) to every record. Finally, a set comprehension over the approved records collects their names.

Exercise 11
Three possible results per pair; use `if / elif / else` inside a comprehension via chained conditional expressions, or a normal loop that appends to a result list. Compare `a` with `b`.

Exercise 12
Build a new dictionary: key is `info["nombre"].upper()`; value is another dictionary with the average, the boolean `avg >= 20`, and the tuple `(max(notas), min(notas))`.

Exercise 13
Write the three lines to the file (one per line, with `\n`). Then read it back, `.strip().split(",")`, convert the numbers with `int()`, compute the average, and collect dictionaries into a list. Finish with `json.dump`.

Exercise 14
The statement says to use the dictionary `datos`, but in the notebook `datos` (Exercise 10) has points, not roles. The CSV layout (`id,nombre,roles`) matches the `usuarios` data of Exercises 5 and 8, so you will probably want a dictionary with ids, names and roles. Since JSON cannot store a set, save roles as a list (Chapter 14). Then reload, uppercase the names, and write the CSV, joining roles with `"|"`.

Exercise 15
Open with `encoding="utf-8"`. Clean and split into words (regex recipe), count with a dictionary or `Counter`, take the 5 most common, and write them with `json.dump` (try `ensure_ascii=False`). Make sure `fragmento-libro.txt` is in the notebook's folder.

Exercise 16
Use a `for` loop over the three file names to write the same content three times. Then read all three, adding each row's quantity to a total per product in a dictionary (convert quantities with `int`). Write the totals to `total_ventas.csv`. With identical files, each total is three times the original value.

Exercise 17
Write the JSON list, read it back, and loop. Compute average, max and min for each student. The statement does not give the pass mark for `"aprobado"`; reusing the rule from Exercise 12 (average ≥ 20) is a sensible choice, and mention that assumption in a comment.

Chapter B

Answers to "Try it yourself"

Chapter 2
(1) `float`. (2) `"55"`, joining text, not adding. (3) `10`.

Chapter 3
(1) `2`. (2) `3`. (3) `True`.

Chapter 4
(1) `'HELLO WORLD'`. (2) `'el'` (indices 1 and 2). (3) `['a', 'b']`.

Chapter 5
(1) `odd` (7 divided by 2 leaves remainder 1). (2) `big` (15 is greater than 10, so the first block runs and the rest are skipped).

Chapter 6
(1) `15`. (2) `[4, 8]`. (3) `4`.

Chapter 7
(1) 3 times (with $i = 0, 1, 2$). (2) `10`.

Chapter 8
`b` is `2`. `t[0] = 9` raises `TypeError` because tuples are immutable.

Chapter 9
(1) `{1, 2, 3}`. (2) `{2, 3}`.

Chapter 10
(1) `{'a': 1, 'b': 2}`. (2) `0`.

Chapter 11
(1) `[0, 1, 4, 9]`. (2) `[4, 5]`.

Chapter 12
(1) `9`. (2) `[3, 2, 1]`. (3) `2.33`.

Chapter C

Quick Reference

Types and literals

Type	Literal	Mutable?
int, float	`5`, `2.5`	no
str	`"text"`	no
bool	`True`, `False`	no
list	`[1, 2, 3]`	**yes**
tuple	`(1, 2, 3)`	no
set	`{1, 2, 3}`	**yes** (unordered, unique)
dict	`{"k": "v"}`	**yes** (key → value)

Collections: what works where

	str	list	tuple	set	dict
ordered / indexable `x[i]`	yes	yes	yes	no	by key
`len(x)`	yes	yes	yes	yes	yes
`for item in x`	yes	yes	yes	yes	keys
`item in x`	yes	yes	yes	yes	keys
slicing `x[a:b]`	yes	yes	yes	no	no

Control flow

```
if cond:            # elif cond2:   else:
    ...
for item in collection:
    ...
for a, b in list_of_pairs:
    ...
for k, v in dictionary.items():
    ...
while cond:
    ...
value = A if cond else B
```

Comprehensions

```
[expr for x in xs]                  # list: transform
[x for x in xs if cond]             # list: filter
[A if cond else B for x in xs]      # list: conditional transform
{k: v for k, v in pairs}            # dict
{expr for x in xs}                  # set
```

Method cheat table

On a…	Useful methods
string	`.upper()` `.lower()` `.capitalize()` `.title()` `.strip()` `.split(sep)` `.replace(a,b)` `.count(x)` `.startswith(x)`; `sep.join(list)`
list	`.append(x)` `.insert(i,x)` `.remove(x)` `.pop()` `.sort()` `.index(x)` `.count(x)`
set	`.add(x)` `.remove(x)` `.update(other)`; operators `|` `&` `-`
dict	`.get(k, default)` `.keys()` `.values()` `.items()` `.setdefault(k, v)` `.update(other)` `.pop(k)`

Files, JSON, CSV in six lines each

```
# --- text ---
with open("f.txt", "w", encoding="utf-8") as f:   f.write("line\n")
with open("f.txt", "r", encoding="utf-8") as f:
    for line in f: print(line.strip())

# --- JSON ---
import json
with open("f.json", "w", encoding="utf-8") as f:  json.dump(obj, f, indent=2)
with open("f.json", "r", encoding="utf-8") as f:  obj = json.load(f)

# --- CSV ---
import csv
with open("f.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f); w.writerow(["a", "b"]); w.writerows(rows)
with open("f.csv", "r", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f): print(row["a"])
```

Final checklist before you submit

- Did you **store** results (`x = x.upper()`) rather than just computing them?
- Are conditions using `==` (not `=`) and are colons and indentation correct?
- Do numbers read from files go through `int()` / `float()`?
- Are sets converted to lists before saving as JSON?
- Did you **Restart & Run All** and confirm the notebook works from top to bottom?
- Are the files (`notas_alumnos.txt`, `resultado.json`, …) created in the notebook's folder with the exact names requested?